ECEN 5113

HW: 1

Part C

Saira Khan

September 10, 2025

Procedure:

Vivado was used to create, synthesize, simulate, and test each design made in this section. An RTL project is created in Vivado, and design sources and simulation files are added to each project. A behavioral simulation is run to generate a waveform (timing schematic). The default FPGA part used is xc7k70tfbv67-1.

A 4-bit ripple carry adder is a digital circuit that adds two four-bit binary numbers by cascading four full adder modules. To create a 4-bit ripple carry adder in Verilog, a full adder module is first created. Figure 2 shows a block diagram of a full adder, which will help in describing the module in Verilog. First, the inputs a, b, and cin (carry in) are declared along with the outputs cout (carry out) and sum. Assign is used instead of an always block because combinational logic is being used. The two outputs, sum and cout, are found using assign statements. The equations are based on logical expressions that can be found using the truth table for a full adder. From there, the 4-bit ripple carry adder can be created. 4-bit inputs A and B are defined along with the input cin, 4-bit output Sum, and cout_f (final carry out). Internal wires are then defined to connect the full adders to each other. Since there are 4 full adders being used to create the 4-bit ripple carry adder, three wires need to be defined to connect them (c1, c2, c3). This will allow for the carry out of the previous full adder to become the carry in of the next full adder. The full adders then need to be instantiated. Instantiating the full adders means that the definition of the full adder, defined before, can be used with different inputs (while the logic remains the same). Four full adders are defined based on the block diagram in Figure 1. A testbench is then created to simulate the logic of the module created. To do this, the inputs/outputs need to be defined, and the 4-bit ripple carry adder is instantiated. An initial block is set up to test different inputs.

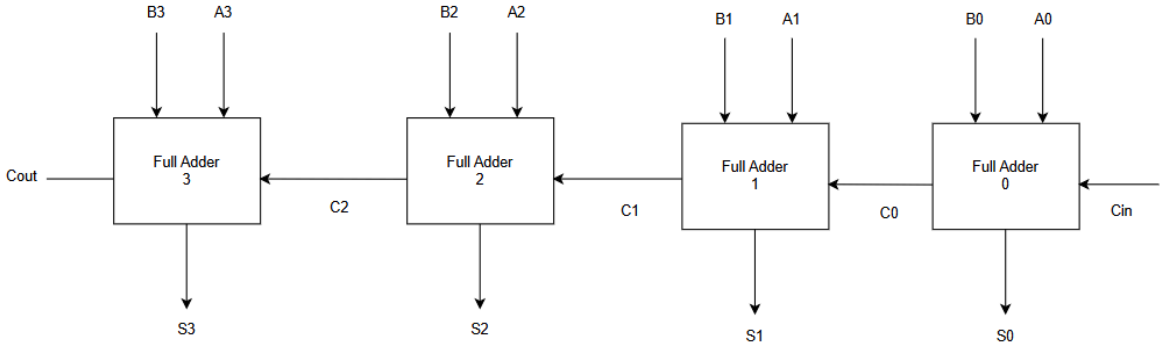

Figure 1: Block diagram of a 4-bit ripple carry adder

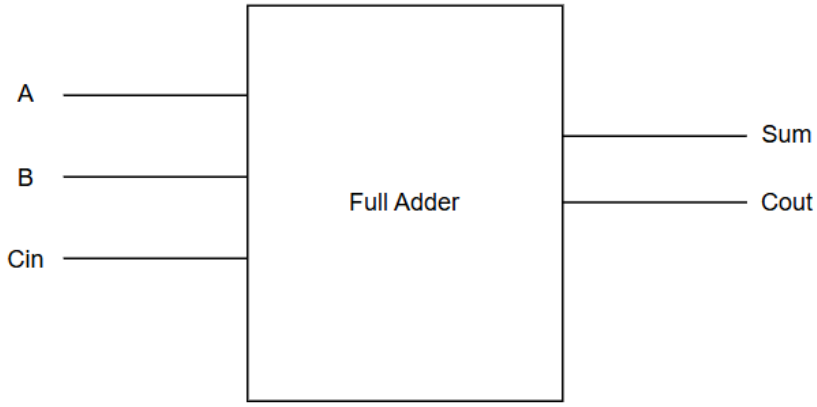

Figure 2: Block diagram of a Full Adder

To create a 32-bit ripple carry adder, eight 4-bit ripple carry (32/4 = 8) adders will be instantiated. Each 4-bit ripple carry adder will handle 4 bits of the addition. The module declares two 32-bit inputs, A and B, an input carry in (cin), a 32-bit output sum, and an output carry out (cout). Carry wires are declared to connect each 4-bit ripple carry adder to the other. The 4-bit ripple carry adders are then instantiated. Since the 4-bit ripple carry adder expects 4-bit inputs and a 4-bit sum, but the 32-bit ripple carry adder has 32-bit inputs/sum, the inputs/sums are bit sliced so that each 4-bit ripple adder gets 4 bits. So A[3:0] means that R0 gets bits 3, 2, 1, and 0 from the 32-bit A. A testbench is created to simulate the logic of the 32-bit ripple carry adder. Inputs/outputs are declared, the module is instantiated, and an initial block is created to test different inputs.

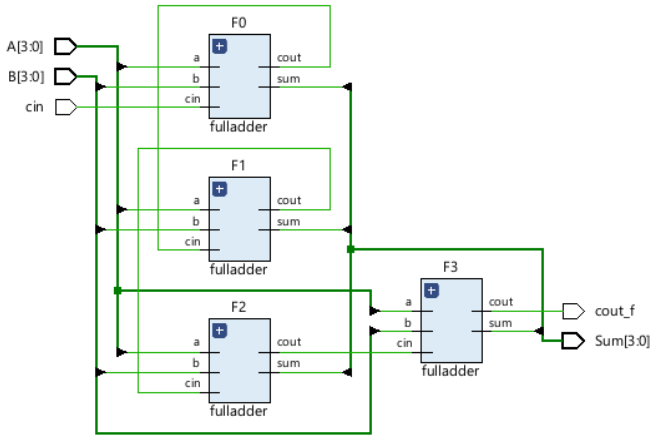

Figure 3: Schematic of 4-bit ripple carry adder

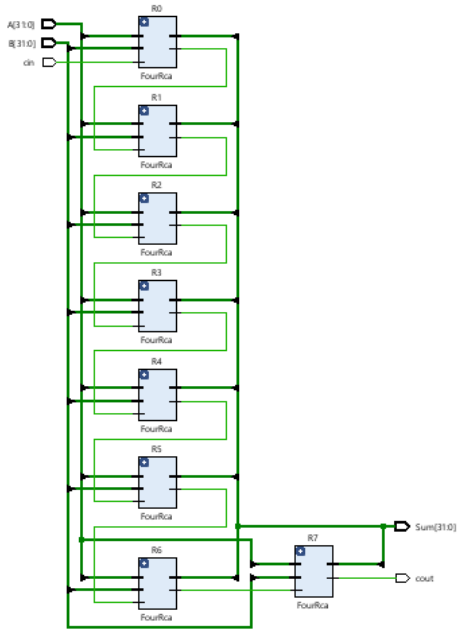

Figure 4: Schematic of 32-bit ripple carry adder

```
//define module
module fulladder (
    //define inputs/outputs
    input a,
    input b,
    input cin,
    output sum,
    output cout
);

    //use assign bc using structural verilog for 4 bit
    //combinational logic
            //calculate the sum
            //use xor (^)
    assign sum = a ^ b ^ cin;

            //calculate the carry out
    assign cout = (a & b) | (cin & (a ^ b));

endmodule

//4-bit ripple carry addermodule
module FourRca(
    //define inputs/output
    input [3:0] A, //4-bit input
    input [3:0] B, //4-bit input
    input cin,//initial carry in
    output [3:0] Sum,//four-bit sum
    output cout_f //final carry out
);
    //internal wires - used to connect the carry-out of one full adder to the carry-in of the next
    wire c1, c2, c3;

    //instantiate four full adders
    fulladder F0 (A[0], B[0], cin, Sum[0], c1);
    fulladder F1 (A[1], B[1], c1, Sum[1], c2); //carry out of F0 becomes carry in for F1
    fulladder F2 (A[2], B[2], c2, Sum[2], c3);
    fulladder F3 (A[3], B[3], c3, Sum[3], cout_f);
endmodule
```

Figure 5: Verilog Code for 4-bit ripple carry adder

```
//testbench module
//define module
module FourRca_tb;

    //declare inputs/outputs
    reg [3:0] A;
    reg [3:0] B;
    reg cin;
    wire [3:0] Sum;
    wire cout_f;


    //instantiate module
    FourRca d(A, B, cin, Sum, cout_f);


    //initial block for set up/simulating
    initial
        begin
        //initialize inputs/outputs
        A = 4'b0000;
        B = 4'b0000;
        cin = 1'b0;

        //testing different inputs
        #10; //wait some time
        A = 4'b0001; B = 4'b0010; cin = 1'b0;
        #1
        $display("time=%0t  A=%4b B=%4b cin=%b | sum=%4b cout=%b",
        $time,A, B, cin, Sum, cout_f);

        #10; //wait 10 time units
        A = 4'b0101; B=4'b0011; cin =1'b0;
        #1
        $display("time=%0t  A=%4b B=%4b cin=%b | sum=%4b cout=%b",
        $time,A, B, cin, Sum, cout_f);

        #10;
        //test carry out
        A = 4'b1111; B =4'b0001; cin =1'b0;
        #1
        $display("time=%0t  A=%4b B=%4b cin=%b | sum=%4b cout=%b",
        $time,A, B, cin, Sum, cout_f);

        #10;
        //test carry in
        A = 4'b0111; B = 4'b0111; cin=1'b1;
        #1
        $display("time=%0t  A=%4b B=%4b cin=%b | sum=%4b cout=%b",
        $time,A, B, cin, Sum, cout_f);

        #10;
        A = 4'b1000; B=4'b1000; cin=1'b0;
        #1
        $display("time=%0t  A=%4b B=%4b cin=%b | sum=%4b cout=%b",
        $time,A, B, cin, Sum, cout_f);

        #10;
        $finish; //end simulation
    end
endmodule
```

Figure 6: Testbench Code for 4-bit ripple carry adder

```
//define module
module rca32 (
    input [31:0] A, //32 bit input A
    input [31:0] B, //32 bit input B
    input cin, //carry input
    output [31:0] Sum, //32 bit sum
    output cout //carry out final
);

    //carry wires
    wire c1, c2, c3, c4, c5, c6, c7;

    //instantiate 8 4-bit rcas
    FourRca R0 (A[3:0], B[3:0], cin, Sum[3:0], c1);
    FourRca R1 (A[7:4], B[7:4], c1, Sum[7:4], c2);
    FourRca R2 (A[11:8], B[11:8], c2, Sum[11:8], c3);
    FourRca R3 (A[15:12], B[15:12], c3, Sum[15:12], c4);
    FourRca R4 (A[19:16], B[19:16], c4, Sum[19:16], c5);
    FourRca R5 (A[23:20], B[23:20], c5, Sum[23:20], c6);
    FourRca R6 (A[27:24], B[27:24], c6, Sum[27:24], c7);
    FourRca R7 (A[31:28], B[31:28], c7, Sum[31:28], cout);

endmodule
```

Figure 7: Verilog Code for 32-bit ripple carry adder

Figure 7 shows the Verilog code for the 32-bit ripple carry adder. What is not shown is the FourRca module and the Full Adder module, which the 32-bit ripple carry adder relies on. These can be seen in Figure 5.

```
//testmodule
//define module
module rca32_tb;

    //declare inputs/output
    reg [31:0] A;
    reg [31:0] B;
    reg cin;
    wire [31:0] Sum;
    wire cout;

    //instantiate module
    rca32 d(A, B, cin, Sum, cout);

    //initial block for simulating
    initial
        begin
        //initalize inputs/output
        A = 32'h00000000;
        B = 32'h00000000;
        cin = 1'b0;

        //testing different inputs
        #10;
        A = 32'h00000000; B = 32'h00000000; cin = 0;
        #1;
        $display("time=%0t  A=%h B=%h cin=%b | sum=%h cout=%b",
        $time,A, B, cin, Sum, cout);
        #10;
        A = 32'h00000001; B = 32'h00000001; cin = 0;
        #1;
        $display("time=%0t  A=%h B=%h cin=%b | sum=%h cout=%b",
        $time,A, B, cin, Sum, cout);
        #10;
        A = 32'hAAAAAAAA; B = 32'h55555555; cin = 0;
        #1;
        $display("time=%0t  A=%h B=%h cin=%b | sum=%h cout=%b",
        $time,A, B, cin, Sum, cout);
        #10;
        A = 32'hFFFFFFFF; B = 32'h00000001; cin = 0;
        #1;
        $display("time=%0t  A=%h B=%h cin=%b | sum=%h cout=%b",
        $time,A, B, cin, Sum, cout);
        #10;

        $finish;
    end
endmodule
```

Figure 8: Testbench Code for 32-bit ripple carry adder

Observations:

For this part, a switch from purely Behavioral Verilog, in Part B, to some structural Verilog was made. Structural Verilog was used because it is easier to model the connections between the full adders. Being able to instantiate the wires that are physically going from one adder to the next helps visualize what the circuit is going to actually look like. Structural modeling has a hierarchical design philosophy that allows designers to make complex structures, such as a ripple carry adder, from simpler components. In this case, using full adders to make a 4-bit ripple carry adder, which was then used to make a 32-bit ripple carry adder.

Results:

4-bit Ripple Carry Adder

Figure 9 shows the waveform simulated by Vivado for the 4-bit ripple carry adder. Looking at one case from 10-20ns where A = 0001 (1) and B = 0010 (2), the waveform shows that the sum will be 3 and that there is no carry output. At 30-40ns the carry out is 1 because there is an overflow. A = 1111 (15) and B = 0001 (1) result in a sum of 0 (black space) and a carry out of 1 (green). Figure 10 shows the log output of the 4-bit ripple carry adder from Vivado. This can be gone through to verify that the output being shown is actually correct.

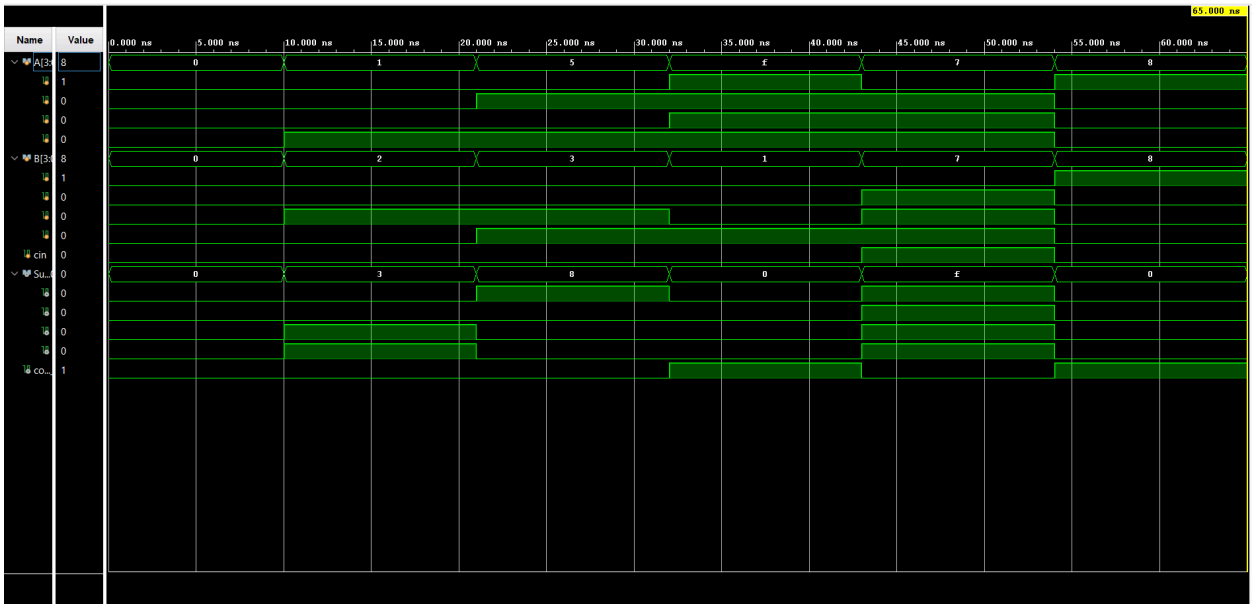

Figure 9: Waveform of 4-bit ripple carry adder

```
time=11000  A=0001 B=0010 cin=0 | sum=0011 cout=0
time=22000  A=0101 B=0011 cin=0 | sum=1000 cout=0
time=33000  A=1111 B=0001 cin=0 | sum=0000 cout=1
time=44000  A=0111 B=0111 cin=1 | sum=1111 cout=0
time=55000  A=1000 B=1000 cin=0 | sum=0000 cout=1
```

Figure 10: Output of 4-bit ripple carry adder

<u>32-bit Ripple Carry Adder</u>

Figure 11 shows the waveform of the 32-bit ripple carry adder simulated on Vivado. From 10ns-20ns, there is an addition of 0s, which is reflected in the sum and carry out. At 30-40s, there is a large amount of green shown by the sum because A and B are large numbers being added together.

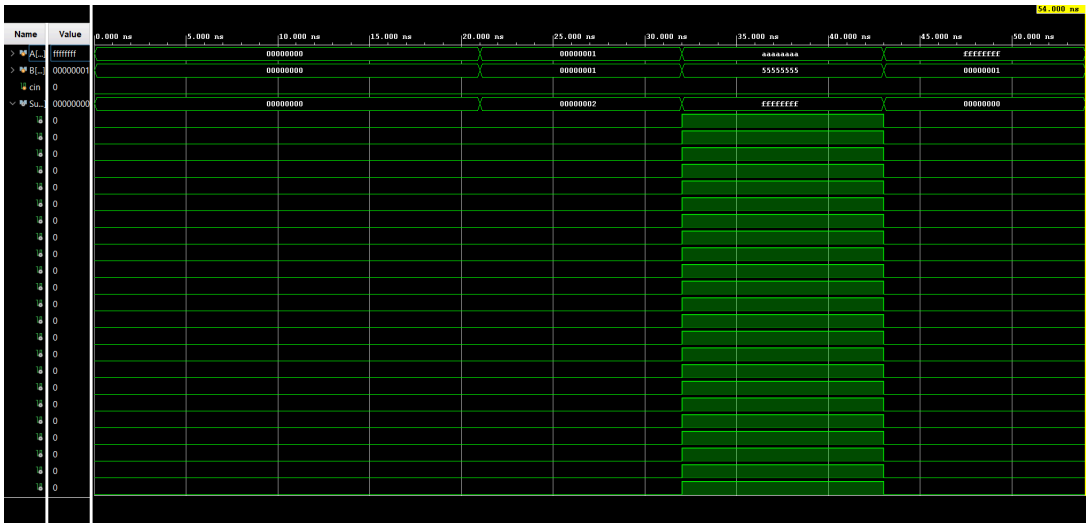

Figure 11: Waveform of 32-bit ripple carry adder

```
time=11000  A=00000000 B=00000000 cin=0 | sum=00000000 cout=0
time=22000  A=00000001 B=00000001 cin=0 | sum=00000002 cout=0
time=33000  A=aaaaaaaa B=55555555 cin=0 | sum=ffffffff cout=0
time=44000  A=ffffffff B=00000001 cin=0 | sum=00000000 cout=1
```

Figure 12: Output of 32-bit ripple carry adder

Figure 12 shows the output log of Vivado when the 32-bit ripple carry adder is simulated. This can be checked over to verify that the logic of the module is working properly.

Conclusion:

Part C demonstrated how complex circuits can be built using smaller, simpler circuits. By instantiating modules, they can be reused in different circuits with different inputs, while keeping the same logic. This makes it easier for designers, helping them not have to write modules over and over again.